

Chapitre 4 exceptions

Synthèse du Chapitre 4 : Les exceptions en PL/SQL

Ce chapitre traite de la gestion des exceptions en PL/SQL, qui permet de gérer les erreurs et les situations imprévues dans les programmes. Les exceptions sont des événements qui interrompent le flux normal d'exécution du programme et nécessitent une gestion spécifique pour éviter des comportements indésirables.

1. Utilisation des exceptions

Les exceptions sont utilisées pour intercepter et gérer les erreurs dans un bloc PL/SQL. Elles peuvent être déclarées, levées explicitement ou implicitement, et gérées à l'aide de gestionnaires d'exceptions.

- **Déclaration des exceptions :**

- Les exceptions sont déclarées dans la section `DECLARE` d'un bloc PL/SQL.
- Syntaxe :

```
Nom_de_l_exception EXCEPTION;
```

- Les exceptions peuvent être levées explicitement avec `RAISE` ou implicitement par Oracle (exceptions prédéfinies).

- **Gestionnaire d'exceptions :**

- Le gestionnaire d'exceptions commence par le mot-clé `EXCEPTION`.
- Il permet d'intercepter les erreurs et de définir des actions spécifiques en fonction du type d'erreur.
- Exemple :

```
BEGIN
    -- Code susceptible de générer une erreur
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Aucune donnée trouvée');
    WHEN OTHERS THEN
```

```
DBMS_OUTPUT.PUT_LINE('Erreur inattendue : ' || SQLERRM);  
END;
```

- **Clause WHEN OTHERS :**

- Cette clause permet d'intercepter toutes les exceptions non gérées par ailleurs.
- Elle est essentielle pour capturer les erreurs inattendues et éviter que le programme ne se termine de manière incontrôlée.
- Exemple :

```
EXCEPTION  
  WHEN OTHERS THEN  
    DBMS_OUTPUT.PUT_LINE('Code Erreur : ' || SQLCODE || ' Mes  
sage : ' || SQLERRM);
```

2. Catégories d'exceptions

Les exceptions en PL/SQL se divisent en plusieurs catégories :

- **Exceptions prédéfinies :**

- Ce sont des exceptions définies par Oracle, associées à des erreurs spécifiques (par exemple, `NO_DATA_FOUND`, `TOO_MANY_ROWS`, `DUP_VAL_ON_INDEX`, etc.).
- Elles n'ont pas besoin d'être déclarées explicitement.
- Exemple :

```
EXCEPTION  
  WHEN NO_DATA_FOUND THEN  
    DBMS_OUTPUT.PUT_LINE('Aucune donnée trouvée');
```

- **Exceptions définies "non redirigées" :**

- Ces exceptions sont définies par le programmeur et doivent être levées explicitement avec `RAISE`.
- Elles ne sont pas associées à une erreur Oracle prédéfinie.
- Exemple :

```

DECLARE
    ExcNomNULL EXCEPTION;
BEGIN
    IF LeNom IS NULL THEN
        RAISE ExcNomNULL;
    END IF;
EXCEPTION
    WHEN ExcNomNULL THEN
        DBMS_OUTPUT.PUT_LINE('Le nom est NULL');
END;

```

- **Exceptions définies "redirigées" :**

- Ces exceptions sont définies par le programmeur mais sont associées à un code d'erreur Oracle via `PRAGMA EXCEPTION_INIT`.
- Elles sont levées automatiquement par Oracle lorsque l'erreur correspondante se produit.
- Exemple :

```

DECLARE
    ExcCleEtrangere EXCEPTION;
    PRAGMA EXCEPTION_INIT(ExcCleEtrangere, -2291);
BEGIN
    -- Code susceptible de générer une erreur de clé étrangère
EXCEPTION
    WHEN ExcCleEtrangere THEN
        DBMS_OUTPUT.PUT_LINE('Violation de contrainte de clé étran
gère');
END;

```

3. Mécanisme d'exécution des exceptions

- Lorsqu'une exception est levée, le contrôle est transféré au gestionnaire d'exceptions du bloc courant.
- Si l'exception n'est pas gérée dans le bloc courant, elle est propagée au bloc englobant ou au programme appelant.

- Si l'exception n'est pas interceptée, le programme se termine avec une erreur.

4. Petit guide pour éviter et intercepter les erreurs

- **Utilisation systématique de `WHEN OTHERS` :**
 - Toujours inclure une clause `WHEN OTHERS` pour capturer les erreurs inattendues.
 - Cela permet de garantir que toutes les erreurs sont gérées, même celles qui n'ont pas été anticipées.
- **Gestion des erreurs spécifiques :**
 - Utiliser des exceptions prédéfinies ou définies pour gérer des erreurs spécifiques.
 - Exemple : gestion des erreurs de conversion de données, des erreurs de contrainte d'intégrité, etc.
- **Validation des données :**
 - Valider les données avant de les utiliser dans des opérations critiques (par exemple, vérifier qu'une date est valide avant de l'insérer dans la base de données).
- **Utilisation de `COMMIT` et `ROLLBACK` :**
 - En cas d'erreur, décider si les modifications doivent être confirmées (`COMMIT`) ou annulées (`ROLLBACK`).
- **Utilisation de `NULL` dans les gestionnaires d'exceptions :**
 - Parfois, il peut être utile de ne rien faire dans un gestionnaire d'exceptions (par exemple, ignorer une erreur spécifique). Dans ce cas, utiliser l'instruction `NULL`.

5. Exemples pratiques

- **Gestion d'une erreur de conversion de nombre :**

```
DECLARE
  ExcNbreInvalide EXCEPTION;
  PRAGMA EXCEPTION_INIT(ExcNbreInvalide, -1722); -- INVALID_NUM
  MBER
```

```

BEGIN
    INSERT INTO Employees (NumSecu, Nom, DateNais) VALUES ('ABC
D', 'Dupont', CURRENT_DATE);
EXCEPTION
    WHEN ExcNbreInvalide THEN
        DBMS_OUTPUT.PUT_LINE('Erreur de conversion de nombre');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Erreur inattendue : ' || SQLERRM);
END;

```

- **Gestion d'une erreur de clé étrangère :**

```

DECLARE
    ExcCleEtrangere EXCEPTION;
    PRAGMA EXCEPTION_INIT(ExcCleEtrangere, -2291); -- Violation de
clé étrangère
BEGIN
    INSERT INTO Employees (NumSecu, Nom, NumChef) VALUES (1, 'Du
pont', 1000); -- NumChef inexistant
EXCEPTION
    WHEN ExcCleEtrangere THEN
        DBMS_OUTPUT.PUT_LINE('Violation de contrainte de clé étrangèr
e');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Erreur inattendue : ' || SQLERRM);
END;

```

Conclusion

La gestion des exceptions en PL/SQL est essentielle pour écrire des programmes robustes et fiables. Elle permet de capturer et de gérer les erreurs de manière contrôlée, évitant ainsi des comportements imprévus et des interruptions brutales du programme. Une bonne pratique consiste à toujours inclure une clause `WHEN OTHERS` pour capturer les erreurs inattendues et à utiliser des exceptions spécifiques pour gérer les erreurs connues.